



Considerações:

1. Trabalho composto por 3 questões valendo 2 pontos extras na Avaliação Parcial 01.
2. Equipes: máximo de 4 alunos. Sendo a responsabilidade : 2 alunos para Questão 1, 1 aluno para Questão 2 e 1 aluno para Questão 3. 80% da pontuação é por desempenho individual e 20% por equipe.
3. Para cada questão, produza uma apresentação com os resultados da simulação. Durante a apresentação, o código correspondente deve estar em condições de execução para demonstração.
4. O material de apresentação deve ser enviado pelo classroom. Os códigos devem ser entregues pelo GitHub no dia da apresentação. Apenas um membro da equipe deve submeter o material na atividade no classroom. Os demais apenas submetem um txt informando o nome do colega que enviou.

Questão 1: [1 ponto] Crie uma simulação que compare diferentes valores de *quantum* para o algoritmo de escalonamento RR (*Round Robin*) com o algoritmo SRTF (*Shortest Remaining Time First*)¹. Dado N processos com diferentes *burst time*², Analise as métricas *tempo médio de resposta*, *tempo médio de retorno* e *vazão* para os diferentes valores de *quantum*. Identifique e demonstre as principais vantagens e desvantagens de ambos algoritmos. Considere:

- Simule tempos de chegadas distintos para os N processos.
- 1 unidade de tempo é gasto para a mudança de contexto do processo. Considere custo de troca de contexto sempre que a CPU muda de um processo para outro, incluindo preempções.
- Em caso de empate, a escolha do escalonador deve ser aleatória, utilizando uma semente fixa (seed) para permitir reprodutibilidade.
- *Tempo médio de resposta (+/- std)*: Tempo médio que um processo leva entre a sua chegada e primeira execução.
- *Tempo médio de retorno (+/- std)*: Tempo médio que um processo leva para ser concluído após sua chegada ao sistema.
- *Vazão* : Quantidade de processos que são concluídos em um determinado período de tempo³.

Exiba a sequência de execução de cada processo para os diferentes valores de *quantum* (RR) e compare com o SRTF. Utilize a linguagem de programação de seu interesse. Demonstre vantagens e desvantagens de ambos algoritmos.

Padronize a entrada do seu algoritmo conforme o json a seguir:

```
{
  "specversion": "1.0",
  "challengeid": "rrsrtfpreemptivo",
  "metadata": {
    "contextswitchcost": 1,
    "throughput window T": 100,
    "algorithms": [ "RR", "SRTF" ],
    "rrquantums": [ 1, 2, 4, 8, 16 ]
  },
  "workload": {
    "timeunit": "ticks",
    "processes": [
```

¹No SRTF, o processo em execução deve ser interrompido sempre que um novo processo chegar com tempo restante menor que o do processo atual.

²Considere gerar tempos de execução pseudoaleatórios em intervalos diferentes de tempo. Exemplo, *burst time* pseudoaleatórios em $[T_1, T_2]$ e *burst time* pseudoaleatórios em $[T_3, T_4]$, com $T_1 < T_2 < T_3 < T_4$. Analise o comportamento dos algoritmos de escalonamento com processos com *burst time* nesses intervalos.

³Por exemplo, quantos processos são concluídos em $T = 100$ unidades de tempo para cada algoritmo de escalonamento abordado? Você pode simular com diferentes valores de T .



```
{
  { "pid": "P01", "arrivaltime": 0, "bursttime": 5 }, { "pid": "P02", "arrivaltime": 1, "bursttime": 17 }, { "pid": "P03", "arrivaltime": 2, "bursttime": 3 }, { "pid": "P04", "arrivaltime": 4, "bursttime": 22 }, { "pid": "P05", "arrivaltime": 6, "bursttime": 7 }
}
```

Questão 2: [1/2 ponto] Cinco programadores estão trabalhando em um laboratório. Cada um está desenvolvendo um módulo diferente de um grande sistema, mas todos precisam compilar seus códigos com recursos limitados: um compilador e um banco de dados de dependências compartilhado.

Cada programador:

- Precisa adquirir acesso exclusivo ao compilador.
- Precisa de acesso compartilhado ao banco de dados, mas apenas dois programadores podem acessá-lo simultaneamente para evitar sobrecarga.
- Após compilar, o programador descansa (pensa) por algum tempo e depois tenta compilar novamente.

Regras:

- Apenas um programador pode usar o compilador por vez.
- No máximo dois programadores podem acessar o banco de dados ao mesmo tempo.
- Um programador só pode começar a compilação quando tiver ambos os recursos.
- O sistema deve evitar deadlocks e inani,ção.

Apresente a saída de forma que seja possível identificar a atividade ou estado de momento de cada programador. Faça com que execute em laço infinito para sua apresentação em sala.

Questão 3: [1/2 ponto] Suponha que um hospital veterinário queira implementar o seguinte protocolo: se um cachorro está na sala de repouso, outros cachorros podem entrar, mas nenhum gato, e vice-versa. Um sinal com uma placa móvel na porta de cada sala de repouso indica em quais dos três estados possíveis a sala pode se encontrar:

- Vazia
- Cachorros presentes
- Gatos presentes

Apresente uma solução com e sem possibilidade de inani,ção.

Padronize a entrada do seu algoritmo conforme o json a seguir:

```
{
  "specversion": "1.0",
  "challengeid": "vetroomprotocoldemo",
  "metadata": {
    "room count": 1,
    "allowedstates": ["EMPTY", "DOGS", "CATS"],
    "signchangelatency": 0,
    "tiebreaker": ["arrivaltime", "id"]
  },
  "room": {
    "initialsignstate": "EMPTY"
  }
}
```

“workload”:{

```
{ "id": "D01", "species": "DOG", "arrivaltime": 0, "restduration": 5 }, { "id":  
  "C01", "species": "CAT", "arrivaltime": 1, "restduration": 4 }, { "id":  
  "D02", "species": "DOG", "arrivaltime": 2, "restduration": 6 }, { "id": "C02",  
  "species": "CAT", "arrivaltime": 3, "restduration": 2 }, { "id": "D03", "species": "DOG",  
  "arrivaltime": 4, "restduration": 3 }  
]  
}
```

Apresenta~oes:

Dia 1 : Quarta-Feira, dia 29 de Abril, `as 7h40.

Dia 2 : Quarta-Feira, dia 29 de Abril, `as 15h50.

